PSEL4 gives H2 users level 3's HQC savings without its regressions

- 15.7 % less HQC than pytket-quantinuum's default level 2 over 217 circuits.
- 0 circuits costlier than level 2 or level 3.
- The lower-HQC compile needs 17.4 % fewer HQC per correct answer on the H2-2E emulator, fewer in 6 of 6 comparisons.
- USD 95,289 to 115,707 saved on the benchmark ledger at Azure plan averages.

Team Learned ZX Rewriting

C.-F. Kam, A. Mahasinghe, D. De Silva, N. Karunarathne, F. Cadet, M. Bessafi, K. De Silva

Quantinuum Singapore Grand Challenge, Submission 1, October 2026

An H2 user pays HQC for every compile choice

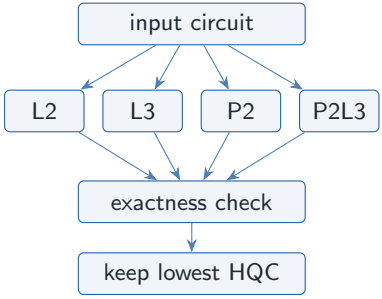
$$\mathrm{HQC} = 5 + C\left(N_{1q} + 10\,N_{2q} + 5\,N_{\mathrm{SPAM}}\right)/5000$$

C is the shot count. N_{1q} counts PhasedX gates, N_{2q} counts ZZPhase and ZZMax gates and N_{SPAM} counts initialisations, measurements and resets. Rz gates are free.

- **User.** A team that runs circuits on H2 under a fixed HQC budget.
- pytket-quantinuum compiles at optimisation level 2 by default.
- Always using level 3 saves 13.9 % HQC, yet costs more than level 2 on 45 of 217 circuits.
- Blind ZX rewriting (PyZX teleport_reduce) before level 2 costs 6.27 % more HQC on 180 Trotter circuits.
- **[O]** The user cannot tell in advance which compile is cheapest for a given circuit.

PSEL4 compiles four ways, checks each and keeps the lowest HQC

- **Candidates.** L2 (default), L3, and PyZX teleport_reduce followed by L2 (P2) or L3 (P2L3).
- **Check.** Full unitary up to 8 qubits, all-zero input state from 9 to 20 qubits. Failed candidates are dropped.
- **Guarantee.** L2 and L3 are always candidates, so PSEL4 never costs more than either where both pass the check.



Drop-in use. One command per circuit file, printing every candidate's HQC and the pick.

```
python3 pipelines/pass_select.py --psel4 <circuit.qasm>
```

Finding 1. Level 3's savings without its regressions

Policy	Total HQC	Saved vs L2	Circuits costlier than L2	HQC lost there
Always L2 (default)	58,837.2			
Always L3	50,632.2	13.9 %	45	372.0
PSEL4	**49,580.6**	**15.7 %**	**0**	**0.0**

217 circuits (A 180, B1 15, C 20, Grover use case 2), HQC at 1000 shots. Source `value/policy_compare.md`, `value/usd_psel4.md`.

- PSEL4 keeps every level 3 saving and adds ZX where it pays. It is 2.08 % below always level 3.
- Always level 3 loses up to 12.95 % on a single circuit (gf2_pow_4_mult). PSEL4 avoids all 45 losses.
- The largest saving is on family C (arithmetic and oracles), 25.8 % below level 2.
- Four compilations take 5.08 times the wall time of level 2 (215 timed circuits), or 3.03 HQC saved per extra second.

Finding 2. 17.4 % fewer HQC per correct answer

Circuit, lower-HQC compile	p level 2	p alt	R	95 % interval of R
vbe_adder_3, P2	0.9520	0.9627	0.7762	0.7690 to 0.7834
hwb6, P2	0.9240	0.9090	0.9730	0.9476 to 0.9991
vbe_adder_7 all-zero input, P2	0.8860	0.8970	0.7851	0.7614 to 0.8095
vbe_adder_7 non-zero input, L3	0.8830	0.9010	0.6114	0.5930 to 0.6303
cdkm_adder_10, L3	0.8692	0.8267	0.9203	0.8896 to 0.9521
modadd_2n_10, P2L3	0.8417	0.8383	0.9526	0.9199 to 0.9864
Pooled, 6 pairs			**0.8262**	**0.8189 to 0.8335**

H2-2E emulator, HQC is the formula estimate. p is the share of shots with the correct output. R is the HQC per correct output of the alt compile over that of level 2. The three newest pairs use the PSEL4 pick. The three earlier pairs use P2, the pick of the earlier two-candidate selector PSEL. grover_5 has no single correct output and is excluded. Source `value/hqc_per_correct.md`.

- R and its 95 % upper bound are below 1 in 6 of 6 pairs.
- On cdkm_adder_10 the level 3 pick returns the correct sum less often ($z = -2.90$) and still costs less per correct sum.

PSEL4's saving is worth USD 95,289 to 115,707 on the benchmark ledger

- Azure plan averages are USD 10.29 (Premium) to 12.50 (Standard) per HQC.
- Against always level 2, PSEL4 saves 9,256.6 HQC, worth USD 95,289 to 115,707.
- Against always level 3, PSEL4 saves 1,051.6 HQC, worth USD 10,825 to 13,145.
- Always level 3 loses USD 3,829 to 4,650 on its 45 costlier circuits. PSEL4 loses nothing there.
- **[O]** Workload assumption: a team spends the Standard Plan's 10,000 HQC a month on this circuit mix. PSEL4 then frees 1,573.3 HQC a month, worth USD 19,666 at the Standard average.

Ledger: 217 circuits, 1000 shots each, HQC formula estimate. Plan averages are not marginal prices and ignore bundled eHQCs and Azure infrastructure costs. Source `value/usd_psel4.md`.

Finding 3. Level 3 cuts two-qubit gates and ZX cuts phases

	Non-Clifford gates				Two-qubit gates			
Set	L2	P2	L3	P2L3	L2	P2	L3	P2L3
B1, 14 circuits	877	656	746	701	862	799	723	700
C, 15 circuits	3323	2930	2540	2535	3208	2944	2718	2708

Totals after each pipeline. 6 circuits did not fit the compile budget. Source `value/mechanism_zx_l3.md`.

- GreedyPauliSimp in level 3 lowers the two-qubit count on 180 of 180 A, 13 of 14 B1 and 9 of 15 C circuits.
- teleport_reduce leaves fewer non-Clifford gates on B1. On H2 this rarely lowers HQC, since a two-qubit gate costs 2 HQC at 1000 shots and a PhasedX 0.2.
- ZX on top of the cheaper pytket level saves 3.21 % on all 20 family C circuits, 0.49 % on B1 and 0 on Trotter circuits.
- On B1 the ZX stage removes 40.6 % of T gates. The Gosset et al. (2014) bound leaves at most 0.3 % for further exact T optimisation (24 circuits of at most 5 qubits).

17 of 20 PSEL4 outputs on family C are proven equal to their input

Family C output	Proven	Not proven	Saved vs L2 (proven)
Level 3	18 of 20	hrs_mult_5, rgqft_mult_6	13.33 %
PSEL4	17 of 20	hrs_mult_5, rgqft_mult_6, draper_adder_28	10.22 %

Family C has 20 to 56 qubits. A PyZX miter reduction proves equality of the full unitary up to global phase. Sources `results/family_C_equivalence_L3.md`, `value/psel4_zx_only.md`.

- On families A, B1 and B2 every compiled circuit matches the uncompiled reference within 1e-8 (1260, 105 and 100 rows).
- Our level 2 pipelines match the official pytket-quantinuum pass on all 445 compiled rows of A, B1, B1x and B2.
- The level 3 miters of the two unproven circuits timed out after 900 s. The PSEL4 miters of rgqft_mult_6 and draper_adder_28 timed out after 300 s.
- hrs_mult_5 has off-grid angles. Its PSEL4 output passes a check on 64 random basis inputs only.

Limitations

- HQC is the formula estimate. All fidelity results come from the H2-2E emulator. Nothing has run on H2 hardware.
- On cdkm_adder_10 the level 3 pick returns the correct sum less often (0.8267 against 0.8692, $z = -2.90$). Accuracy-bound users need a fidelity-aware choice.
- The ZX-only gain is 0.26 % on the 17 proven family C circuits.
- 2 family C level 3 outputs and 3 PSEL4 outputs are unproven. On draper_adder_28 the PSEL4 pick starts from angles PyZX rounded by up to 1.50e-06 rad.
- The earlier vbe_adder_3 fidelity gain used the all-zero input. Its Holm-adjusted p is 0.052 over three circuits.
- No user interview or survey is part of this submission. The selector follows pytket's PassSelector pattern.

Finals plan is a fidelity-aware learned selector

- **[O]** Goal: a learned selector that predicts fidelity as well as HQC for each candidate. It keeps the per-answer saving and protects accuracy where users need it.
- cdkm_adder_10 shows why. Its cheaper compile costs less per correct sum but returns the correct sum less often.
- A first classifier on input-circuit features does not beat constant choices with a whole family held out (balanced accuracy 0.433 against 0.500).
- **[O]** The finals model therefore needs structural features and emulator labels.
- **[O]** Its starting data are the four-candidate ledger of 217 circuits and the 34 stored H2-2E jobs.

Every result can be rerun from the code package

- `code.zip`. The source of the repository. `README.md` gives the run instructions.
- `requirements.txt`. Pins pytket 2.18.4, pytket-quantinuum 0.59.3 and pyzx 0.10.7.
- `run_tests.sh`. Runs 35 test files and writes one return code per file to `submission/build_logs/tests_summary.txt`.
- `EVIDENCE.md`. Lists every emulator job with its job ID, shots, HQC estimate and payload. The 34 H2-2E jobs used an HQC estimate of 8983.4 of the 10,000 budget.
- **Generated tables.** Value tables come from the scripts under `value/`. Emulator tables are regenerated offline from the stored payloads.
- **Independent checks.** Verification reports by agents that did not produce the results are in `results/`.